GEN AI-POWERED PERSONALIZED MENTAL HEALTH SUPPORT SYSTEM

19ITP401 PROJECT I REPORT

Submitted by

DEEPAK C	**-**	**713522IT013**
MADHUMITHA S	**-**	**713522IT033**
ADHITHYAN L	**-**	**713522IT501**
PRADEEP KUMAR S	**-**	**713522IT503**

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

i*n*

INFORMATION TECHNOLOGY

SNS COLLEGE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

DECEMBER 2025

BONAFIDE CERTIFICATE

Certified that this Project Report titled **"GEN AI-POWERED PERSONALIZED MENTAL HEALTH SUPPORT SYSTEM"** is this Bonafide work of

1. **DEEPAK C (713522IT013)**
2. **MADHUMITHA S (713522IT033)**
3. **ADHITHYAN L (713522IT501)**
4. **PRADEEP KUMAR S (713522IT503)**

who carried out the work under my supervision for the year 2025. Certified further that to the best of my knowledge the work reported herein has been completed in all aspects and necessary documents have been submitted to the concerned in-change of the department.

PROJECT GUIDE	**HEAD OF THE DEPARTMENT**
Mr. K. S. MOHAN	**Dr. S. ANGEL LATHA MARY**
Assistant Professor, Department of IT, SNS College of Technology, Coimbatore-641035.	Professor & Head of the Department, Department of IT & AIML, SNS College of Technology, Coimbatore-641035.

Submitted for the Viva-Voce examination held at SNS College of Technology, on……

Internal Examiner External Examiner

ABSTRACT

In recent years, artificial intelligence has shown significant potential in enhancing mental health support systems through natural language understanding and personalized interaction. However, existing platforms often lack context awareness and adaptability when providing emotional assistance. To address this gap, **nMinds** introduces a **Retrieval-Augmented Generation (RAG)-based personalized mental health support platform** that leverages **Generative AI** to deliver contextually grounded and empathetic responses.

The system is built on a **modular microservice architecture**, where the **backend (developed in Go)** functions as the primary orchestrator for handling API requests, database communication, and AI model integration. The **frontend, developed using Next.js and React**, provides an intuitive and responsive user interface for seamless conversational interaction. The core intelligence of the system is powered by **Gemini**, an advanced language model that performs contextual reasoning and personalized dialogue generation.

A crucial component of the architecture is the **RAG pipeline**, which transforms uploaded documents or contextual data into embeddings stored within **ChromaDB**, a vector database integrated with a traditional relational database (Postgres/MySQL). During a user query, the system retrieves the most relevant knowledge segments to augment the prompt, enabling the AI model to produce responses that are both contextually accurate and emotionally relevant. With a focus on modularity and scalability, **Phase One** of nMinds implements the foundational components—RAG integration, AI response generation, and backend orchestration—serving as the groundwork for subsequent phases that will include workflow automation, sentiment analysis, and multimodal input/output capabilities. This phase demonstrates the potential of combining **RAG architecture with Generative AI** to create a reliable, context-aware, and scalable mental health support system.

ACKNOWLEDGEMENT

First and foremost, we extend our heartfelt gratitude to the **Management of SNS College of Technology** for providing us with all the necessary support and facilities to successfully complete this project.

We express our sincere thanks to our **Principal, Dr. S. Chenthur Pandian**, for his valuable guidance and constant encouragement throughout the course of this project.

We are deeply grateful to **Dr. R. Sudhakaran, Final Year Cluster Head CS Stream, Dr. L. M. Nithya, Professor & Dean (CSE, IT & AIML), Dr. S. Angel Latha Mary, Professor & Head (IT & AIML) and Dr. S. Narayanan, IV Year HoD (IT & AIML),** for their valuable suggestions, guidance, and positive support extended to us from the beginning to the successful completion of this project.

We express our sincere appreciation to our **Project Co-ordinator, Ms. Padmapriya R, Assistant Professor, Department of IT** for his continuous encouragement and support in carrying out this project effectively.

We take immense pleasure in expressing our profound gratitude to our **Project Guide, Mr. K. S. Mohan, Assistant Professor, Department of IT**, for her remarkable guidance, insightful suggestions, and constant motivation, which helped us complete the project successfully and on time.

Finally, we would like to extend our heartfelt thanks to all the **Faculty Members, Technical Staff, Parents, and dear Friends** for their moral support and encouragement throughout the completion of this **Project–I**.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	i
	ACKNOWLEDGEMENT	ii
	LIST OF FIGURES	v
1	**INTRODUCTION**	1
	1.1 OVERVIEW	1
	1.2 PROBLEM STATEMENT	4
	1.3 PURPOSE OF PROJECT	5
	1.4 DOMAIN OVERVIEW	7
	1.4.1 GO	7
	1.4.2 NEXT.JS	8
	1.4.3 VECTOR EMBEDDINGS AND SEMANTIC SEARCH	8
	1.4.4 Large Language Models (LLMs)	9
	1.4.5 ChromaDB (Vector Database)	9
	1.4.6 JWT (JSON Web Token) Authentication	9
	1.4.7 Database Management (Postgres/MySQL	10
2	**LITERATURE SURVEY**	11
3	**WORK DONE IN PHASE I**	14
	3.1 EXISTING SYSTEM	14
	3.2 PROPOSED SYSTEM	15

	3.2.1 WORK DONE FOR PHASE I	15
	3.3 ARCHITECTURE OF THE PROPOSED SYSTEM	16
	3.4 WORKFLOW DIAGRAM	18
4	**SYSTEM REQUIREMENTS**	19
	4.1 OUTPUT	22
	4.2 ALGORITHM	24
5	**CONCLUSION AND FUTURE WORK**	27
	5.1 CONCLUSION	27
	5.2 FUTURE WORK	28
6	**APPENDIX 1**	29
	A1.1 SOFTWARE SPECIFICATION	29
	APPENDIX 2	31
	A2.1 SOURCE CODE	31
	APPENDIX 3	33
	A3.1 SAMPLE RAG CODE	33
7	**REFERENCES**	41

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
3.4	**Workflow**	18
4.1.1	**UI/UX page 1**	22
4.1.2	**UI/UX page 2**	22
4.1.3	**UI/UX page 3**	23
4.1.4	**UI/UX page 4**	23
A2.1.1	**Frontend CSS Code**	31
A2.1.2	**Frontend HTML Code**	31
A2.1.3	**Backend Code**	32
A2.1.4	**Backend Code**	32

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

In an era where mental well-being is increasingly recognized as a critical part of personal and professional life, technology has begun to play a vital role in offering accessible and personalized mental health support. However, many existing digital solutions lack contextual understanding, personalized memory, and dynamic information retrieval. To address these limitations, **nMinds** introduces a **Retrieval-Augmented Generation (RAG)-based mental health assistance system** designed to provide informed, empathetic, and contextually relevant responses using **Generative AI**.

The goal of nMinds is to create an intelligent platform capable of understanding user inputs, retrieving related knowledge from a curated dataset, and generating personalized responses through **Gemini**, an advanced AI model. The platform emphasizes simplicity and modularity in **Phase One**, focusing primarily on the integration of a **RAG pipeline**, a **Go-based backend**, and a **Next.js + React frontend**.

The Role of AI in Mental Health Support

Artificial intelligence has transformed how mental health resources can be delivered at scale. Through context-aware natural language processing and retrieval-based reasoning, AI systems can assist individuals by offering personalized coping strategies, relevant literature, and conversational support. The **RAG pipeline** ensures that generated responses are not only conversationally coherent but also grounded in factual and therapeutic context drawn from trusted sources.

In this phase, the system operates primarily as a **text-based knowledge and support interface**—retrieving relevant insights from embedded data stored in **ChromaDB** and generating grounded responses via **Gemini**. Future phases will extend these capabilities to include sentiment analysis, automation workflows (via n8n), and multimodal interaction.

Core Features of nMinds (Phase One)

1. **Document Upload & RAG-Powered Query**
   Users can upload or access curated documents, which are processed into vector embeddings using advanced text embedding models. These are stored within **ChromaDB**, integrated with a relational database (Postgres/MySQL). When a user inputs a query, the system performs **semantic similarity search** to identify relevant passages, which are then provided to the **Gemini AI model** for context-augmented response generation.
   This ensures factual accuracy and helps reduce hallucination in responses—making the platform suitable for educational, therapeutic, and knowledge-based applications.

2. **Conversational AI Interface**
   The **Next.js + React** frontend provides an intuitive chat-like interface, enabling users to interact seamlessly with the AI system. Each conversation request is sent to the backend through RESTful APIs, processed by the **Go microservice**, and routed through the RAG pipeline for enhanced answer generation.
   The platform maintains a minimal context memory per session to ensure fluid interactions and easy scalability in future phases.

3. **Backend and RAG Architecture**
   The **backend, developed in Go**, acts as the main orchestrator responsible for handling API requests, user session management, database operations, and integration with the **Gemini API**.
   The **RAG pipeline** is modular, with distinct services for:
   - Document processing and chunking
   - Embedding generation
   - Vector storage in **ChromaDB**
   - Context retrieval and query augmentation

4. **Data Management and Storage**
   The system uses **Postgres/MySQL** for structured data and **ChromaDB** for vector embeddings, ensuring both traditional relational operations and fast semantic search capabilities. Data flow between components is managed via REST APIs, with strict validation and JWT-based authentication for secure user interactions.

Technical Implementation and Architecture

Backend Technology Stack

- **Language:** Go (Golang)
- **Architecture:** Modular microservice with controllers, services, and database layers
- **Core Components:**
    - Controllers for handling requests (RAG, conversation)
    - Services for business logic and Gemini integration
    - Database models and query handlers
    - Middleware for authentication and request validation

Frontend Technology Stack

- **Framework:** Next.js with React
- **Styling:** Tailwind CSS for modular and responsive design
- **Features:**
    - Chat interface for text interaction
    - API integration with backend endpoints
    - State management using React hooks and context

AI Integration

- **Model:** Gemini API for conversational response generation
- **Vector Store:** ChromaDB for semantic retrieval and knowledge grounding
- **Database:** Postgres/MySQL for structured data storage

Authentication & Security

- **Authentication:** JWT-based session control
- **Data Protection:** Secure handling of user input and stored embeddings

Evaluation Metrics and Success Criteria (Phase One)

1. **Retrieval Accuracy:**
   Measures how effectively the RAG pipeline retrieves relevant content for a given query.
2. **Response Quality:**
   Evaluates coherence, contextual grounding, and factual reliability of AI-generated answers.
3. **System Performance:**
   Monitors backend latency, query processing time, and concurrent request handling capability.
4. **User Experience:**
   Assessed through usability testing—simplicity, clarity, and relevance of responses.

1.2 PROBLEM STATEMENT

- In the current digital environment, individuals seeking mental health support often encounter generic, impersonal, or unreliable online responses. Many platforms lack the ability to understand emotional context or provide evidence-based, personalized guidance.
- Most conversational AI systems are trained on general datasets and are not grounded in verified psychological or well-being content, leading to potential misinformation or emotionally insensitive outputs.
- There is also a shortage of AI-driven systems that integrate document-based reasoning for mental health support. Users and professionals often maintain their own curated materials (articles, therapy guides, self-help notes), but lack tools that can query and retrieve insights contextually from these sources.
- Traditional chatbots and therapy support apps rely on static responses or keyword matching instead of context-aware retrieval and response generation. This limits the system's ability to adapt to unique user inputs or remember previous sessions.

- Furthermore, managing and scaling such a platform is technically challenging without modular architecture. Integrating multiple AI capabilities—retrieval, understanding, generation, and memory—requires robust backend orchestration, secure data handling, and efficient vector search mechanisms.
- This project aims to address these challenges by building nMinds, a Retrieval-Augmented Generation (RAG)-based mental health support platform that combines knowledge-grounded retrieval, conversational intelligence, and contextual understanding through Gemini AI, supported by a Go backend, Next.js + React frontend, and ChromaDB vector store.
- The goal is to create an accessible, accurate, and empathetic AI system capable of assisting users in self-reflection, emotional understanding, and knowledge-driven mental health guidance — all within a scalable and privacy-preserving architecture.

1.3 PURPOSE OF PROJECT

Objective

The primary goal of **nMinds** is to develop an intelligent and context-aware conversational platform for **personalized mental health support**, powered by **Retrieval-Augmented Generation (RAG)**. In this **Phase One**, the system focuses on text-based interactions that retrieve contextually relevant responses from stored knowledge sources and generate empathetic, grounded replies through the **Gemini AI model**.

Core Features and Functionalities (Phase One)

1. **Retrieval-Augmented Conversational AI (RAG)**
   The system integrates a RAG pipeline where documents or curated mental health content are converted into vector embeddings and stored in **ChromaDB**. When users ask a question or share their thoughts, the system performs semantic search to retrieve the most relevant context and provides accurate, evidence-based responses via the **Gemini API**.

2. **Context-Aware Conversation**
   Through minimal session memory, **nMinds** maintains conversational continuity, allowing follow-up questions to build on prior exchanges. This enhances emotional coherence and personalization within conversations.

3. **Scalable Backend Architecture**
   The **Go-based backend** acts as the central orchestrator, handling API requests, user management, RAG operations, and secure communication with the Gemini model. This modular setup allows for seamless upgrades in later phases, including automation and multimodal support

4. **Interactive Frontend Interface**
   The **Next.js + React** frontend provides a chat-like experience for smooth user interactions. Its lightweight, responsive design ensures accessibility on web and mobile browsers.

5. **Data Management and Vector Retrieval**
   The system uses a combination of **Postgres/MySQL** (for structured data) and **ChromaDB** (for vector embeddings). This hybrid structure ensures both traditional query support and high-speed semantic retrieval.

Technologies and Techniques Used

- **AI Model:** Gemini API (Generative AI for conversational reasoning)
- **Vector Store:** ChromaDB for retrieval and contextual grounding
- **Backend:** Go (Golang) – RESTful architecture for performance and scalability
- **Frontend:** Next.js + React for interactive UI
- **Database:** Postgres/MySQL for structured data storage
- **Authentication:** JWT-based security and session control

Evaluation Metrics

1. **Retrieval Accuracy:**
   The system's ability to identify and return contextually relevant responses from the stored embeddings.

2. **Response Quality:**
   Evaluates coherence, emotional tone, and factual grounding of AI-generated answers.

3. **System Performance:**
   Includes API response time, retrieval speed, and concurrent session handling.

4. **User Experience:**
   Based on simplicity, empathy, and relevance of interactions during evaluation sessions.

Vision for Future Phases

Phase Two and beyond will expand upon this foundation by integrating:

- **Sentiment analysis nodes (via n8n)** for emotion recognition.
- **Voice input/output** for multimodal interaction.
- **DeepSearch pipelines** for refined multi-LLM reasoning.
- **Multi-answer exploration** for presenting diverse perspectives to users.

Together, these enhancements will transform **nMinds** from a retrieval-based assistant into a truly multimodal, emotionally adaptive AI companion for mental wellness.

1.4 DOMAIN OVERVIEW

1.4.1 Go (Golang)

Go, also known as Golang, is a statically typed, compiled programming language created by Google engineers Robert Griesemer, Rob Pike, and Ken Thompson. It is widely recognized for its simplicity, performance, and concurrency model, making it ideal for scalable backend systems, microservices, and distributed architectures.

Go's lightweight concurrency primitives (goroutines and channels) allow developers to handle thousands of simultaneous operations efficiently, while its clean syntax and fast compilation enhance

productivity and code maintainability.

In **nMinds**, Go serves as the **core backend language**, responsible for handling user sessions, orchestrating API requests, managing data flow between the relational database (Postgres/MySQL) and ChromaDB, and communicating with the Gemini AI model.

It provides the foundation for performance, scalability, and modular integration with external AI services, making it an ideal choice for a context-aware, real-time conversational system.

1.4.2 Next.js

Next.js is a React-based framework developed by Vercel that enables developers to build highly optimized, modern web applications. It supports server-side rendering (SSR), static site generation (SSG), and client-side rendering, offering flexibility for performance-critical applications.

Its **file-based routing**, **API routes**, and **built-in optimization** features allow seamless integration between frontend and backend components, enabling faster data fetching and dynamic rendering.

In this project, **Next.js** serves as the **frontend framework**, providing an intuitive interface where users can chat, upload documents, and receive personalized mental health insights. The chat interface interacts with the Go backend to process user inputs and display responses from the Gemini model in real time.

Next.js ensures a responsive, accessible, and smooth user experience across devices.

1.4.3 Vector Embeddings and Semantic Search

Vector embeddings are dense, numerical representations of text that capture semantic meaning. In high-dimensional vector space, semantically similar texts are placed closer together.

In **nMinds**, embeddings are crucial for enabling **Retrieval-Augmented Generation (RAG)**. When a user submits a query, the system:

1. Converts stored document chunks into vector embeddings using embedding models.
2. Stores them in **ChromaDB**, a vector database optimized for semantic search.
3. Converts the user's query into a vector and finds the closest matches using similarity search (cosine similarity).
4. Feeds the retrieved context to the **Gemini model** to produce accurate, grounded, and emotionally sensitive responses.

This process ensures that responses are contextually relevant, evidence-based, and tailored to the user's knowledge base or mental health resources.

1.4.4 Large Language Models (LLMs)

Large Language Models (LLMs) are transformer-based AI models trained on massive text corpora to understand, generate, and reason over human language. They can perform tasks like question answering, summarization, paraphrasing, and emotional reasoning.

In this project, **Gemini** (Google's advanced multimodal LLM) serves as the **core conversational intelligence engine**.

It performs:

- **Conversational reasoning:** maintaining context across turns to engage in meaningful dialogue.
- **Retrieval-Augmented Generation:** combining document knowledge with its own understanding to generate grounded responses.
- **Emotional awareness:** providing empathetic and supportive replies based on user inputs.

Gemini's long context window and reasoning ability allow **nMinds** to generate coherent and context-aware guidance aligned with psychological best practices.

1.4.5 ChromaDB (Vector Database)

ChromaDB is a high-performance open-source vector database designed for semantic search and AI applications. It efficiently stores embeddings and supports approximate nearest neighbor (ANN) search to retrieve relevant information in real time.

In **nMinds**, ChromaDB powers the RAG pipeline by:

- Storing embeddings derived from user documents or curated wellness datasets.
- Performing vector similarity search to retrieve conceptually relevant content.
- Enabling hybrid search through integration with Postgres/MySQL metadata (user, timestamp, document category).

This combination ensures **fast**, **scalable**, and **context-aware** retrieval of knowledge that grounds the AI's responses in reliable data.

1.4.6 JWT (JSON Web Token) Authentication

JWT (JSON Web Token) is a compact, secure token standard used for stateless authentication. It encodes user identity and session data, signed cryptographically to prevent tampering.

In **nMinds**, JWT ensures that only authenticated users can access sensitive routes such as document upload, RAG queries, or chat sessions.

The authentication workflow includes:

1. User login and verification.
2. Token issuance from the Go backend.

3. Token validation on each subsequent API request.
4. Controlled access to AI and RAG services.

This stateless mechanism allows scalability without the overhead of session management while maintaining user security and privacy.

1.4.7 Database Management (Postgres/MySQL)

A relational database such as **PostgreSQL** or **MySQL** serves as the backbone for structured data storage in **nMinds**.

It stores user profiles, chat histories, document metadata, and authentication records.

Key advantages include:

- **Data Integrity:** ACID compliance ensures reliable data transactions.
- **Query Efficiency:** Optimized indexing supports fast data retrieval.
- **Integration with Vector Store:** Works alongside ChromaDB to maintain hybrid data search (relational + semantic).

This dual-database architecture (SQL + vector) ensures both **precision and flexibility** in information retrieval.

CHAPTER 2

LITERATURE SURVEY

The field of conversational AI has evolved rapidly, moving from simple, stateless chatbots to more sophisticated systems capable of context-aware reasoning. The integration of Large Language Models (LLMs) has greatly enhanced fluency and coherence, but challenges remain in maintaining long-term memory, grounding responses in external knowledge, and delivering personalized interactions. This survey reviews research in memory architectures, retrieval-augmented generation, and vector-based knowledge systems that inform the design of modern conversational platforms, particularly those supporting mental health and knowledge-based applications.

2.1 Evolution Toward Hybrid Memory Architectures

Early conversational systems were limited by short-term memory, restricting their ability to maintain context over extended interactions. Modern research emphasizes **hybrid memory architectures**, which manage multiple types of information essential for human-like dialogue:

- **Semantic Memory:** Stores factual knowledge and domain-specific information using vector embeddings for efficient semantic retrieval.
- **Episodic Memory:** Maintains a chronological record of past user interactions to support context-aware responses.
- **Procedural Memory:** Encodes step-by-step knowledge for guiding users through multi-step tasks or structured workflows.

Vector databases like ChromaDB provide a practical foundation for semantic memory. However, challenges such as data consistency, completeness, and drift highlight the need for careful embedding management and validation to ensure high-quality retrieval.

In **nMinds Phase One**, semantic and episodic memory are prioritized:

- Semantic memory is implemented through ChromaDB to index knowledge from mental health resources.
- Episodic memory is maintained in Postgres/MySQL for conversation history, allowing context-aware responses from Gemini.

2.2 Retrieval-Augmented Generation (RAG) for Dynamic Knowledge

Retrieval-Augmented Generation (RAG) addresses the limitations of LLMs' static knowledge by grounding responses in external sources. A retriever module fetches relevant information, which is then used to guide the LLM's output, improving factual accuracy and relevance.

Research by Zhang et al. suggests a dual-layered approach:

- **Plan A (Expert Retrieval):** Queries are first matched against curated, domain-specific document collections.
- **Plan B (Fallback LLM):** If the retrieved content is insufficient, the LLM generates responses based on pre-trained general knowledge.

In **nMinds**, the RAG pipeline retrieves mental health insights and conversation-relevant context from ChromaDB to feed Gemini, ensuring responses are **accurate, contextually relevant, and empathetic**.

2.3 Context-Aware Conversational AI

Maintaining conversation context across multiple turns is critical for meaningful interactions. Modern AI platforms implement memory tracking, session management, and context injection to enhance dialogue coherence.

Key techniques include:

- **Session-based context storage:** Retaining recent conversation turns.
- **Memory nodes for hybrid storage:** Combining long-term knowledge with short-term conversation data.
- **Context injection into prompts:** Ensures that LLM responses are grounded in user-specific data.

nMinds Phase One leverages these techniques to provide a conversational agent that:

- References prior interactions within a session.
- Uses document context retrieved via RAG to answer queries accurately.
- Delivers responses tuned for supportive and empathetic engagement.

2.4 Building a Knowledge-Driven Conversational Platform

The concept of a **personalized knowledge base** is central to modern conversational systems. Components include:

- **Document embeddings:** Convert user-uploaded documents into semantic vectors.
- **Chunking and indexing:** Break documents into manageable segments for efficient retrieval.

- **Contextual retrieval:** Rank chunks by relevance for situational awareness.

This approach allows the platform to provide responses grounded in both user-specific and curated content. For mental health applications, this ensures that advice or insights are evidence-based and appropriately contextualized.

2.5 Summary of Findings

The literature highlights the following trends:

1. **Hybrid memory architectures** improve coherence by managing semantic, episodic, and procedural information.
2. **RAG systems** enable dynamic grounding of LLMs in user-specific or domain-specific knowledge.
3. **Context injection and session management** are essential for coherent, empathetic conversations.
4. Knowledge bases structured with **vector embeddings** provide fast, semantically relevant retrieval while supporting personalization.

For **nMinds Phase One**, these insights validate the design choices:

- Using ChromaDB for semantic memory.
- Maintaining conversation logs in Postgres/MySQL.
- Leveraging Gemini LLM with RAG to generate grounded, context-aware, empathetic responses.

These foundations provide a robust framework for future enhancements, including multimodal content generation, automated workflows, and richer personalization in Phase Two.

CHAPTER 3

WORK DONE IN PHASE I

3.1 EXISTING SYSTEM

The contemporary landscape of conversational AI and content generation is dominated by platforms leveraging Large Language Models (LLMs) for text-based interactions. Many employ Retrieval-Augmented Generation (RAG) to ground responses in external knowledge via vector databases. While these systems improve on traditional rule-based chatbots, they still have limitations:

- **Monolithic memory architecture:** Most systems rely solely on vector embeddings, struggling to manage semantic, episodic, procedural, and emotional memory types.
- **Out-of-Distribution (OOD) vulnerability:** Queries outside the model's knowledge base or document corpus can result in hallucinations or inaccurate answers.
- **Limited multimodal integration:** Many platforms are text-focused and lack cohesive pipelines for images, audio, or video, often producing fragmented outputs.
- **Scalability challenges:** Processing entire documents or large files in real-time requires high computation, making many solutions unsuitable for responsive, user-facing applications.

3.1.1 Drawbacks of Existing Systems

- **Lack of hybrid memory architectures:** Existing systems rarely differentiate between long-term factual knowledge and short-term conversation history. This reduces multi-turn coherence and personalization.
- **Semantic drift:** Vector databases are prone to inconsistencies over time, causing identical queries to yield varying results.
- **Poor OOD handling:** Single-expert AI models fail gracefully when faced with queries outside their trained corpus.
- **Inefficient multimedia pipelines:** One-shot generation methods lack iterative refinement, leading to suboptimal quality in generated media.
- **Resource-intensive workflows:** Without preprocessing or chunking, document and media processing can be slow and computationally expensive.

3.2 PROPOSED SYSTEM

To overcome these challenges, **nMinds Phase I** focuses on a modular conversational AI platform with document-grounded RAG and future-ready multimodal support. The system leverages a microservices-style Go backend, a Next.js/React frontend, and ChromaDB for semantic memory.

Key design innovations include:

- **Plan A – Plan B RAG Framework:**
  - **Plan A (Expert AI):** Retrieves context from the vector index built from user-uploaded documents.
  - **Plan B (General AI):** Falls back to Gemini for general-purpose responses when retrieval is insufficient.
- **Hybrid Memory Architecture:**
  - **Semantic Memory:** Managed via ChromaDB vector store containing document embeddings.
  - **Episodic Memory:** Stored in Postgres/MySQL, maintaining conversation logs for multi-turn context.
- **Modular Orchestration:** Go backend acts as the orchestrator, managing calls to Gemini, SDXL, and other generative services through a late fusion strategy.
- **Scalable and Extensible Architecture:** Containerized services (Docker) enable independent scaling of microservices and future integration of additional models or modalities.

3.2.1 Work Done in Phase I

Phase I focused on building the foundational platform for nMinds, prioritizing document-grounded chat and modular AI orchestration. The core tasks completed are:

1. Backend Architecture

- Implemented Go-based microservices-style backend with controllers, services, and models.
- Exposed RESTful APIs for authentication (JWT), user management, and content generation.
- Integrated asynchronous processing for scalable, non-blocking operations.

2. RAG & Database Integration

- Connected ChromaDB for storing document embeddings and semantic retrieval.
- Implemented pipelines for document upload, chunking, and embedding storage.
- Python microservice handles embedding generation and similarity search efficiently.

3. AI Workflow & n8n Integration

- Integrated n8n for workflow automation, routing tasks like RAG queries and image generation.
- Established webhook-based communication between Go backend and n8n.
- Event-driven triggers enable intelligent routing to appropriate model pipelines.

4. Model Integrations

- Gemini integrated as primary conversational LLM.
- SDXL linked for text-to-image generation.
- TTS models prepared for audio output (planned for Phase II).

5. Frontend Development

- Built Next.js/React frontend for chat and content generation interfaces.
- Integrated real-time chat with optional voice input/output.
- Designed modular UI components for document upload, chat history, and generated outputs.

6. Containerization & Deployment

- Dockerized backend and microservices for portability.
- Configured environment variables and service isolation.
- Prepared preliminary deployment with Docker Compose for future scalability.

3.3 ARCHITECTURE OF THE PHASE I SYSTEM

The system is a **decoupled frontend-backend application** with modular services and containerized deployment:

Frontend (Next.js/React)

- Provides intuitive interfaces for document upload, chat, and content generation.
- Communicates with backend via REST API.
- Renders generated text, images, and audio outputs.

Backend (Go)

- **Controllers:** Handle HTTP requests and JWT authentication.
- **Services/Orchestrator:**
    - Manages calls to Gemini and other services.
    - Implements "Plan A – Plan B" RAG logic.
- **Models/Database:** Stores metadata for users, documents, and generated content.

Multimodal Knowledge Base (MMKB)

- **File Storage:** Stores raw documents and generated media as Multimodal Nodes (MMNs).

- **Vector Database (ChromaDB):** Stores embeddings for semantic retrieval.
- **Chunking Pipeline:** Breaks large files into smaller units for efficient retrieval and "frame ranking" of relevant content.

Data Flow

1. **Perception:** User interacts via frontend (upload, chat, prompt).
2. **Orchestration:** Request routed to backend controller.
3. **Comprehension:** Service layer retrieves semantic and episodic memory; prepares model prompts.
4. **Projection:** Calls Gemini or SDXL; stores generated outputs.
5. **Rendering:** Frontend displays response or provides download/playback of media.

3.4WORKFLOW DIAGRAM

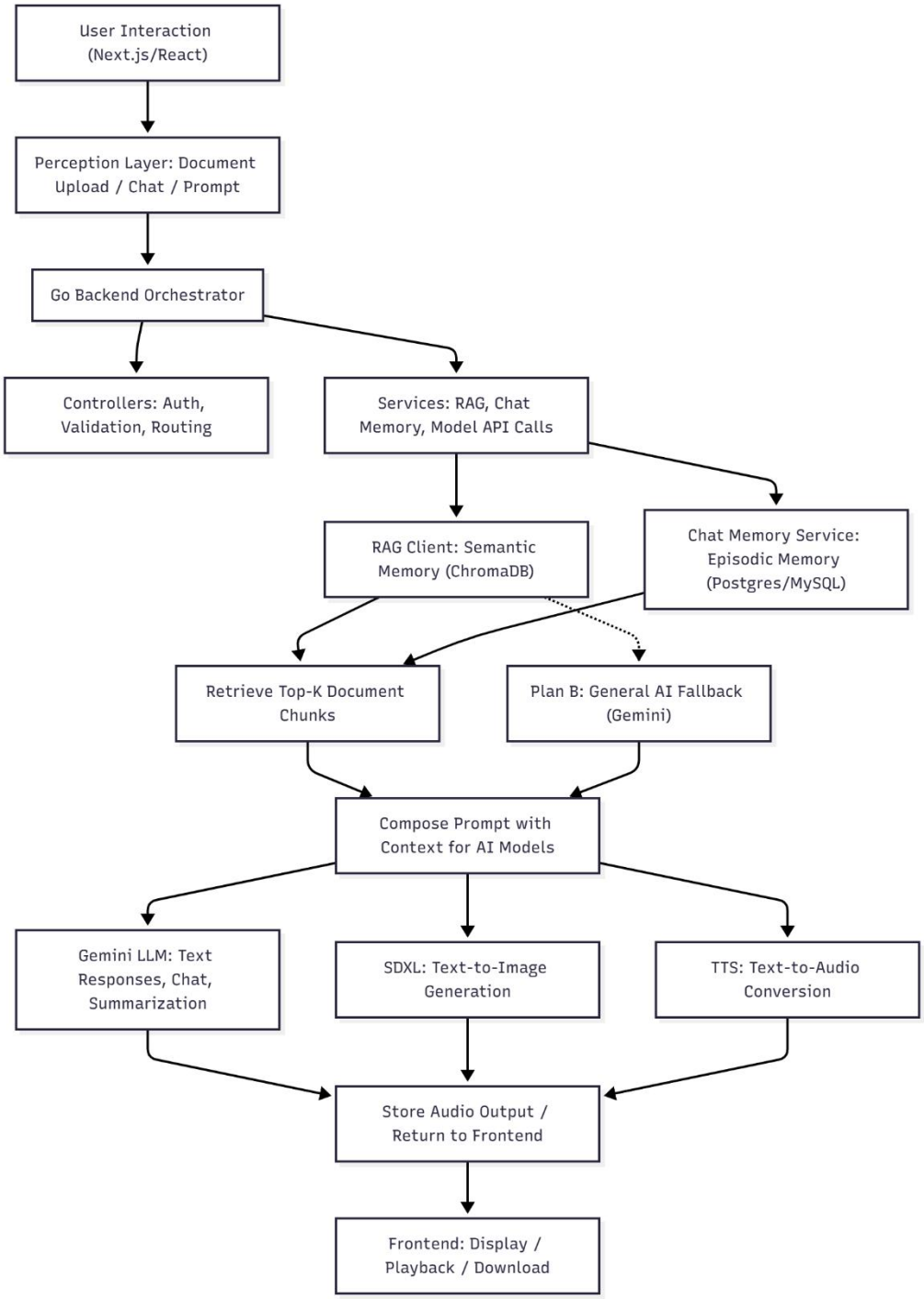

Fig 3.4.1 workflow

CHAPTER 4

SYSTEM REQUIREMENTS

The **NMinds** system is an intelligent mental health conversational agent designed to provide contextual, empathetic, and accurate responses through a **Retrieval-Augmented Generation (RAG)** setup. In Phase I, the system focuses on building a strong foundation for document-based context retrieval, semantic memory management, and conversational understanding through the **Gemini API**. Artificial Intelligence (AI) plays a central role in enabling **context-aware dialogue**. By leveraging **vector embeddings**, **semantic retrieval**, and **document grounding**, NMinds can deliver personalized, relevant, and emotionally appropriate responses. This initial phase provides a scalable backend framework that will later integrate advanced automation and multimodal features in future stages.

System Requirements Summary

- **Frontend:** Next.js + React (simple chat interface with document upload)
- **Backend:** Go (REST API with modular architecture)
- **Database:** PostgreSQL/MySQL (for structured data and chat logs)
- **Vector Store:** ChromaDB (for RAG-based semantic retrieval)
- **AI Model:** Gemini API (for generating context-aware responses)
- **Deployment:** Dockerized services for backend and vector database
- **Authentication:** JWT-based user authentication
- **Memory Design:** Hybrid (Semantic via ChromaDB, Episodic via SQL)

List of Modules

1. Document Ingestion & Processing
2. Vector Embedding Generation
3. Semantic Retrieval & RAG Pipeline
4. Conversational Orchestration
5. Evaluation & Quality Assurance

Module Descriptions

1. Document Ingestion & Processing

This module allows users to upload mental health or wellness-related documents that the system will use to ground responses. Uploaded files are processed through:

- **File validation** (type, size, encoding)
- **Text extraction and cleaning**
- **Chunking** (splitting into manageable sections for embedding)
- **Metadata tagging** (document title, type, and user reference)

The processed text and metadata are stored for embedding generation.

Implemented in: backend/controllers/ and backend/services/.

2. Vector Embedding Generation

Each document chunk is converted into a high-dimensional embedding to represent its meaning numerically.

This enables **semantic search** rather than keyword-based retrieval.

Features:

- Batch embedding generation for efficiency
- Normalization and storage in **ChromaDB**
- Caching for re-use of previously embedded content
- Integration through the RAG service module (ragclient.go)

3. Semantic Retrieval & RAG Pipeline

This is the **core intelligence layer** of NMinds Phase I.

It retrieves the most contextually relevant passages from ChromaDB and uses Gemini to generate a grounded response.

Steps:

1. **User query embedding** → transformed via same model used for document embeddings.
2. **Similarity search** → top relevant chunks fetched from ChromaDB.
3. **Prompt assembly** → query + retrieved context combined.
4. **Gemini API call** → generates grounded, conversational response.
5. **Response return** → to the frontend with contextual understanding.

This module ensures that NMinds behaves like a clinically aligned chatbot with accurate, document-backed reasoning.

4. Conversational Orchestration

The backend orchestrates all data flow between the frontend, vector store, and Gemini model.

It handles:

- Request validation

- Memory retrieval (episodic conversation logs)
- Context preparation and response routing
- Logging and error handling

This Go-based orchestration ensures modular design, allowing future integration with voice, workflows, and sentiment analysis nodes.

5. Evaluation & Quality Assurance

To ensure response quality and consistency:

- **Retrieval metrics**: precision and recall of RAG retrieval.
- **Response accuracy**: checks for factual consistency and relevance.
- **Performance metrics**: API latency, response time, and system throughput.
- **User feedback logging** (for iterative improvement).

In Phase I, evaluation focuses on **accuracy of retrieved context** and **response coherence**, ensuring the foundation is robust for Phase II enhancements.

4.1.Output

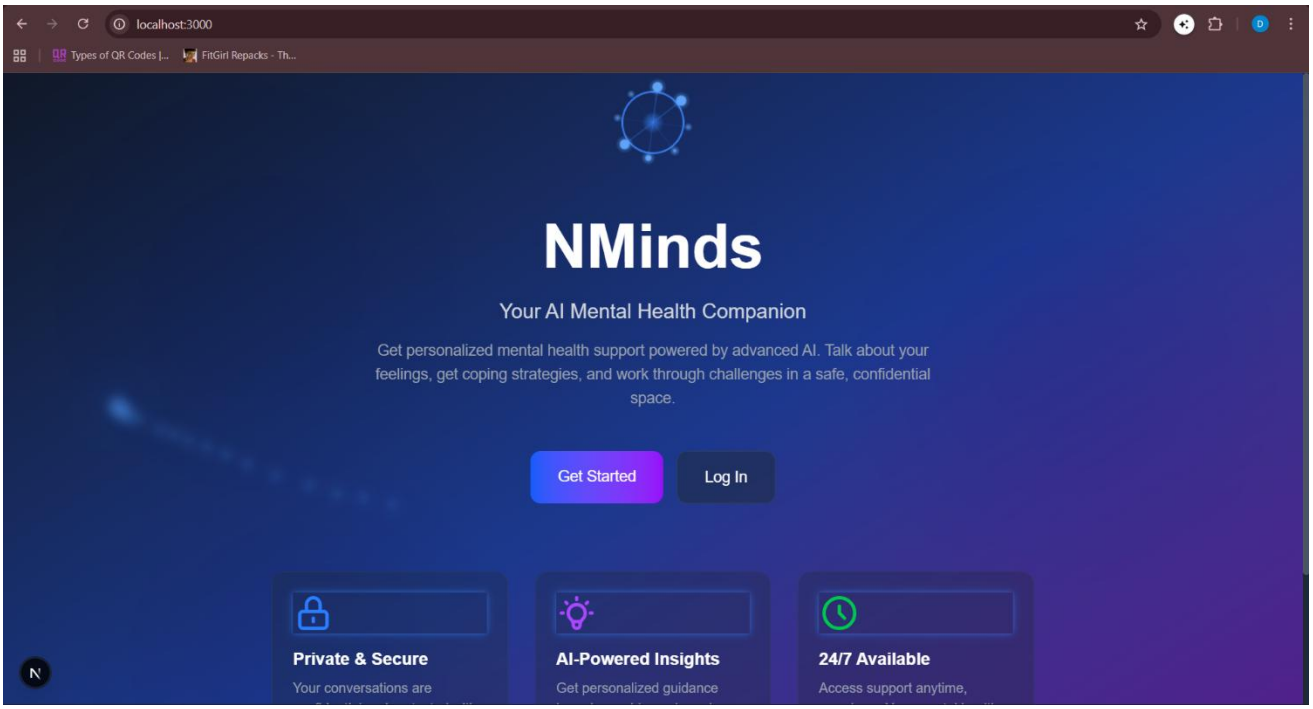

Fig 4.1.1 UI/UX

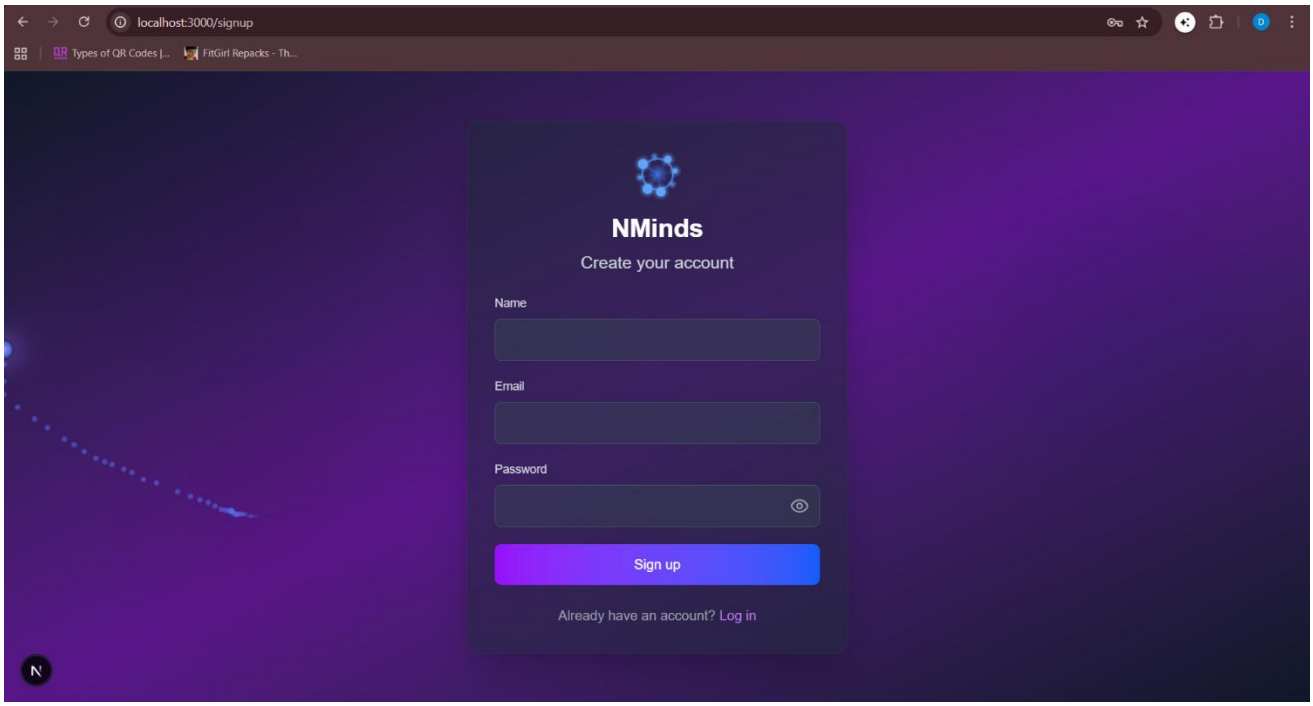

Fig 4.1.2 UI/UX page 2

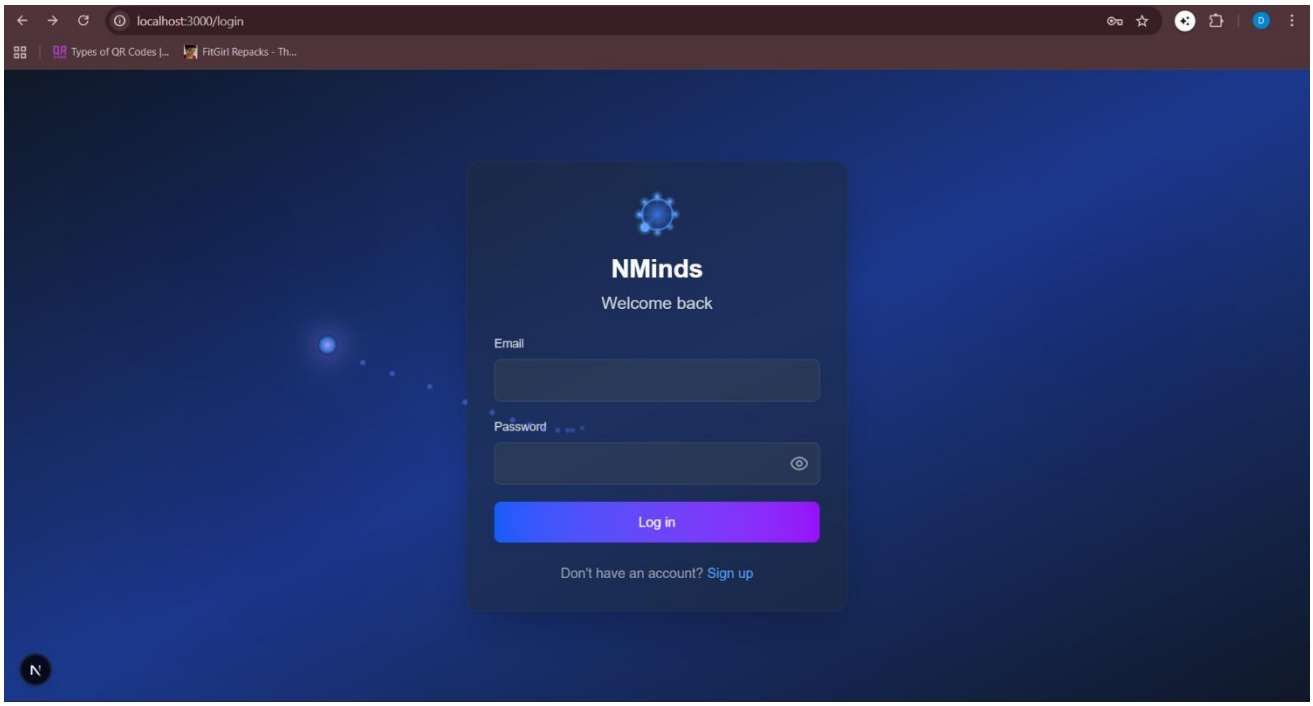

Fig 4.1.3 UI/UX page 3

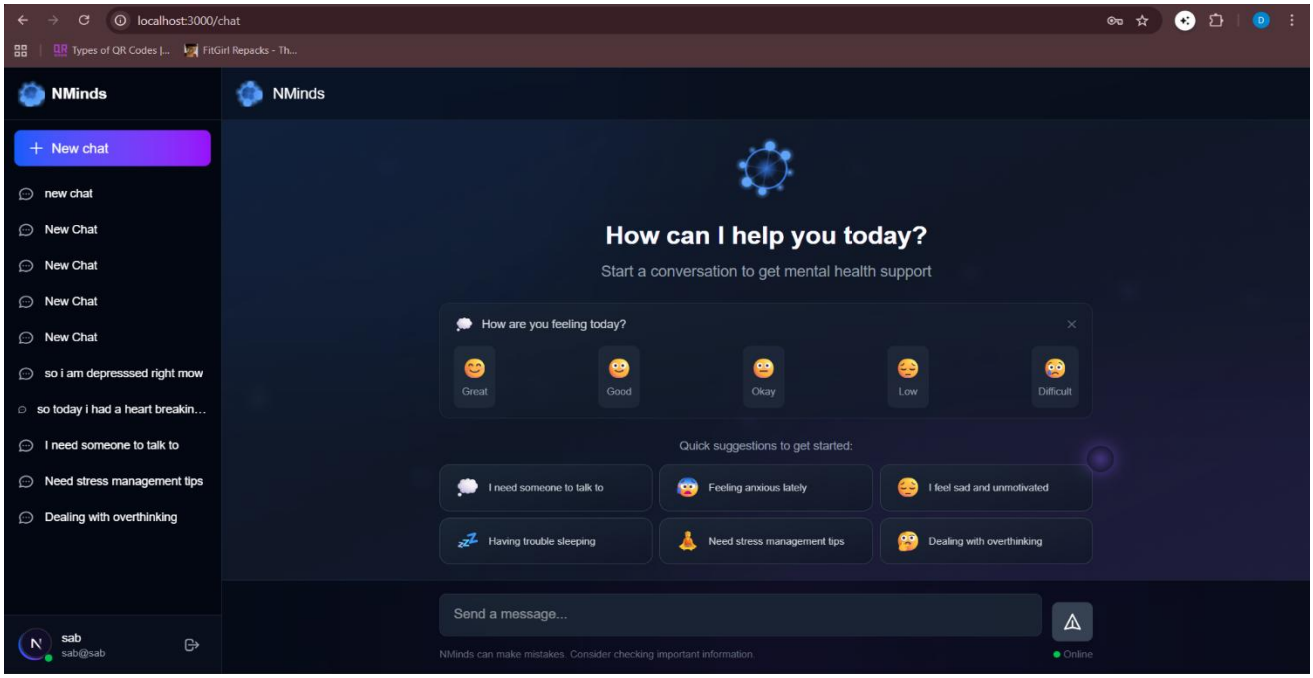

Fig 4.1.4 UI/UX page4

4.2 ALGORITHM

The proposed NMinds system integrates retrieval-augmented conversational intelligence with context-grounded text generation. It leverages transformer-based large language models (LLMs) and dense vector retrieval mechanisms to deliver accurate, context-aware responses based on user-provided documents.

Phase One focuses solely on text-based RAG (Retrieval-Augmented Generation) functionality with a Go backend, Gemini API for AI responses, and ChromaDB + SQL hybrid memory for retrieval and conversation history.

1. Large Language Model (LLM – Gemini API)

Algorithm: Transformer-based Deep Learning Architecture

Purpose:

The system employs Google's **Gemini API**, a transformer-based large language model, to process user queries and generate conversational responses. The LLM applies multi-head self-attention and positional encoding mechanisms to understand semantic relationships within text, enabling coherent and contextually relevant responses.

When user prompts are received, the backend constructs a contextual input containing both the user message and previously retrieved knowledge snippets, ensuring continuity and factual grounding.

2. Retrieval-Augmented Generation (RAG)

Algorithm: Dense Vector Retrieval + Contextual Prompt Composition

Purpose:

This is the core of NMinds Phase One. Uploaded knowledge documents are processed, chunked, and stored as vector embeddings in **ChromaDB**. Each chunk captures semantic meaning using embedding generation models compatible with Gemini's retrieval format.

Steps:

1. The user query is converted into an embedding vector.
2. The system performs a **similarity search** against ChromaDB using **cosine similarity** to identify the most semantically relevant document chunks.
3. The retrieved context is appended to the user's query.
4. The combined prompt is sent to the Gemini API, which generates a context-grounded response.

This mechanism ensures answers are both semantically accurate and contextually consistent with previously stored or uploaded data.

3. Hybrid Memory Architecture

Algorithm: Dual Memory Storage (Semantic + Episodic)

Purpose:

NMinds uses a hybrid memory system to retain both factual and conversational data:

- **Semantic Memory:** Vector embeddings stored in **ChromaDB** for document retrieval.
- **Episodic Memory:** Conversation logs stored in **SQL (Postgres/MySQL)**, preserving session continuity and enabling context-aware chat history reconstruction.

This dual-structure approach allows the system to recall relevant past interactions and maintain consistent context across multiple user sessions.

4. Backend Service Orchestration

Algorithm: Modular Controller–Service Pattern in Go

Purpose:

The backend, built with **Go**, follows a modular design where each controller handles user requests and delegates logic to service layers.

- **RAG Client Service:** Manages vector search, context assembly, and Gemini API requests.
- **Conversation Service:** Handles session tracking and memory updates.
- **Database Service:** Manages both SQL storage (episodic) and ChromaDB operations (semantic).

This design ensures high scalability, clean separation of logic, and maintainability for future upgrades.

5. Security and Access Control

Algorithm: JWT Authentication + Role-Based Access Control

Purpose:

User sessions are protected using **JWT (JSON Web Token)** authentication. Tokens verify identity and ensure only authorized users can initiate or continue conversations. Secure APIs protect data exchange between the frontend and backend, with environment-based API key management for Gemini integration.

6. Frontend Communication

Algorithm: API-driven Client–Server Interaction

Purpose:

The **Next.js + React** frontend communicates with the Go backend through REST APIs. Each query

triggers a pipeline:

1. User Input → Frontend API Request
2. Backend → RAG Retrieval + Gemini Generation
3. Response → Frontend Display (with session context)

This architecture ensures responsiveness and real-time updates while maintaining modular separation between presentation and intelligence layers.

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

The **NMinds Phase One** project successfully establishes the foundation for an intelligent, context-aware mental health conversational agent powered by **Retrieval-Augmented Generation (RAG)**.
This phase focuses on building a robust, text-based backend that integrates the **Gemini API** for language understanding and generation, a **Go backend** for logic and orchestration, a **Next.js + React frontend** for user interaction, and a **hybrid memory system** combining **SQL (episodic)** and **ChromaDB (semantic)** layers.

The system can perform document-grounded, context-aware chat, maintaining conversation history and delivering meaningful responses based on uploaded or pre-stored knowledge. By adopting a modular and API-driven design, NMinds ensures scalability, security, and clear separation between frontend, backend, and AI services.

This implementation demonstrates a functional prototype capable of RAG-based knowledge grounding, conversation memory, and Gemini-powered reasoning — providing a strong, production-ready core for future extensions such as multimodal features, sentiment analysis, and automation workflows.

In summary, **NMinds Phase One** achieves:

- A fully functional backend using Go with RAG and Gemini API integration
- Hybrid memory architecture with ChromaDB and SQL
- Context-aware conversational AI with knowledge grounding
- Secure communication using JWT authentication
- Scalable and modular architecture for future expansion

This foundation positions NMinds as a reliable platform to evolve into a complete mental health companion with advanced capabilities in subsequent phases.

5.2 Future Work

The next phase of NMinds will expand its capabilities beyond text-only RAG into multimodal intelligence, automation, and therapeutic augmentation. The planned enhancements include:

- **Sentiment Analysis & Therapeutic Contexts:**
  Integration with n8n workflows for real-time emotion detection, clinical schema generation, and context-based therapeutic responses.
- **Voice & Speech Integration:**
  Adding **Whisper (speech-to-text)** and **Kokoro/F5-TTS (text-to-speech)** modules to support multilingual, natural voice interactions.
- **Human-in-the-Loop Escalation:**
  Implementing conservative escalation rules for critical risk detection (e.g., suicidal ideation) with clinician-style audit trails and manual verification paths.
- **Workflow Automation via n8n:**
  Connecting NMinds to external systems (Notion, Slack, etc.) through automated workflows for journaling, reminders, and daily check-ins.
- **Advanced RAG Features:**
  Introducing **Graph-based RAG** and **DeepSearch refinement** to enhance long-term memory, contextual continuity, and retrieval precision.
- **Scalability & Monitoring:**
  Implementing async task queues for embedding generation, load balancing, and monitoring dashboards for system observability and reliability.
- **Security & Compliance:**
  Expanding JWT-based authentication to include **RBAC (Role-Based Access Control)**, encryption for stored data, and improved audit logging.

CHAPTER 6

APPENDIX 1

A1.1 SOFTWARE SPECIFICATION

Operating System

- **Development Environment:** Windows 10 / 11
- **Deployment Environment:** Docker Containers

Frontend

- **Framework:** Next.js (React + TypeScript)
- **Styling:** Tailwind CSS
- **Tools & Libraries:**
  - Node.js
  - npm / yarn
  - Axios (API communication)
  - VS Code (Development IDE)

Backend

- **Language:** Go (Golang)
- **Frameworks & Libraries:**
  - net/http – HTTP handling
  - gorm – ORM for database operations
  - jwt-go – Authentication and authorization
- **Architecture:**
  - Modular structure with Controllers, Services, Models, and Middleware
- **APIs:** RESTful endpoints for chat, memory, and RAG (Retrieval-Augmented Generation)

AI & Model Services

- **LLM:** Gemini API – Core conversational reasoning and response generation
- **RAG System:** ChromaDB (Vector Database) for semantic context retrieval
- **Future Expansion (Phase Two):**
  - n8n for workflow automation and sentiment routing
  - Whisper (STT), Kokoro/F5-TTS (Speech synthesis)
  - ComfyUI for visual generation workflows

    - DeepSearch and Multi-Answer options for enhanced reasoning

Data & Storage

- **Database:** PostgreSQL / MySQL (for metadata, user sessions, and chat logs)
- **Vector Store:** ChromaDB (stores embeddings for semantic retrieval)
- **File Storage:** Local directory (/backend/media/) for document or RAG assets

Integration Layer

- Provides communication between the backend (Go) and the Gemini API
- Handles context-aware request-response flow with memory persistence

Development & Monitoring Tools

- **Docker / Docker Compose:** For containerized deployment and environment consistency
- **Postman:** API endpoint testing
- **Git & GitHub:** Version control and repository management
- **Google Colab / Jupyter:** (For future experimentation and model testing)

Packages & Dependencies

- **Go Backend:**
    - gorm, jwt-go, net/http
- **Frontend (Next.js):**
    - react, axios, next, tailwindcss
- **Python-side (for future integration):**
    - chromadb, pandas, numpy, matplotlib, requests, fastapi

APPENDIX 2

A2.1 SOURCE CODE

FRONTEND

```
function ChatPageContent() {
  async function sendTranscribedMessage(transcript) {
      const data = await res.json();
      if (!res.ok) throw new Error(data.error || "Speech request failed");
      const newConvId = data.conversation_id;
      if (!activeChatConversationId && newConvId) {
        setActiveChatConversationId(newConvId);
        try { localStorage.setItem("lastChatConversationId", newConvId); } catch {}
        updateUrlParams({ c: newConvId });
      }
      const convIdToFetch = newConvId || activeChatConversationId;
      if (convIdToFetch) {
        const full = await fetch(`${API_BASE}/conversations/${convIdToFetch}/messages`, { headers: { ...authHeaders() } }).then((r) => r.json());
        setChatMessages(Array.isArray(full) ? full : []);
      } else {
        setChatMessages((prev) => [
          ...prev,
          { id: `ai-${Date.now()}`, role: "model", content: data.reply || "", timestamp: new Date().toISOString() },
        ]);
      }
    } catch (e) {
      setChatMessages((prev) => [
        ...prev,
        { id: `err-${Date.now()}`, role: "model", content: `Error: ${e.message}`, timestamp: new Date().toISOString() },
      ]);
    } finally {
      setSendingChat(false);
    }
  }
  function renderMain() {
    return (
      <section className="flex h-full flex-1">
        {/* Messages column: scrollable independently */}
        <div ref={messagesRef} className="flex-1 overflow-y-auto p-4 pb-28" style={{ height: 'calc(100vh - 96px)' }}>
          <div className="mx-auto flex w-full max-w-4xl flex-col gap-4">
            {chatMessages.length === 0 && (
              <div className="mt-16 rounded-2xl border □border-slate-800/60 □bg-black/30 p-8 text-center ■text-slate-400">
                Start a conversation with the neon assistant
              </div>
            )}
            {chatMessages.map((m) => (
              <MessageBubble key={m.id || m.timestamp} msg={m} />
            ))}
          </div>
        </div>
```

Fig A2.1.1 Frontend code

```
frontend
  src
    app
      agentChats
      chat
        page.jsx                U
      deep-research
      flux
      flux_kontext
      library
      login
      music-gen
      profile
      rag
      sdxl
      signup
      video-gen
      voice-gen
      # globals.css            M
      layout.jsx              U
      page.jsx                U
    components
    hooks
  .dockerignore               U
  .gitignore
  compose.yaml                U
  Dockerfile                  U
  {} jsconfig.json            M
  JS next.config.mjs
  {} package-lock.json        M
  {} package.json             M
  JS postcss.config.mjs
  README.Docker.md            U
  ⓘ README.md                 M
  JS tailwind.config.js       U
  API_DOCS - Copy.md
  API_DOCS.md
  Project -1 Team 4.docx
```

```
export default function Landing() {
                <div>
                  <div className="text-sm font-medium ■text-slate-200">Conversation exported</div>
                  <div className="text-xs ■text-slate-400">2 hours ago</div>
                </div>
              </div>
              <div className="flex items-start gap-3">
                <div className="h-9 w-9 rounded-lg □bg-sky-900 flex items-center justify-center ■text-sky-300">RG</div>
                <div>
                  <div className="text-sm font-medium ■text-slate-200">New collection created</div>
                  <div className="text-xs ■text-slate-400">Yesterday</div>
                </div>
              </div>
            </div>
          </motion.div>
        </div>
      </section>

      {/* Features - staggered cards with icon left */}
      <section id="features" className="mx-auto w-full max-w-6xl px-6 pb-16">
        <div className="grid gap-6 sm:grid-cols-2 lg:grid-cols-3">
          {features.map((c, i) => (
            <motion.div
              key={i}
              initial={{ opacity: 0, y: 18 }}
              whileInView={{ opacity: 1, y: 0 }}
              viewport={{ once: true, amount: 0.35 }}
              transition={{ duration: 0.55, delay: i * 0.06 }}
              whileHover={{ y: -6 }}
              className="flex gap-4 items-start rounded-2xl border □border-slate-800/60 □bg-black/30 p-5"
            >
              <div className="flex h-12 w-12 items-center justify-center rounded-lg bg-gradient-to-br □from-sky-800/20 □to-sky-900/10 ■text-sky-300">
                {c.icon}
              </div>
              <div>
                <h3 className="text-sm font-semibold ■text-sky-300">{c.title}</h3>
                <p className="mt-1 text-sm ■text-slate-300/80">{c.body}</p>
              </div>
            </motion.div>
          ))}
        </div>
      </section>
```

Fig A2.1.2 Frontend code

BACKEND

```
backend > controllers > auth.go > SignupInput > Password
```

```go
package controllers

import (
    "backend/database"
    "backend/models"
    "backend/utils"

    "github.com/gofiber/fiber/v2"
)

type SignupInput struct {
    Username string `json:"username"`
    Email    string `json:"email"`
    Password string `json:"password"`
}

type LoginInput struct {
    Email    string `json:"email"`
    Password string `json:"password"`
}

func Signup(c *fiber.Ctx) error {
    var input SignupInput
    if err := c.BodyParser(&input); err != nil {
        return c.Status(fiber.StatusBadRequest).JSON(fiber.Map{"error": err.Error()})
    }
    hashedPassword, err := utils.HashPassword(input.Password)
    if err != nil {
        return c.Status(fiber.StatusInternalServerError).JSON(fiber.Map{"error": "Failed to hash password"})
    }
    user := models.User{Username: input.Username, Email: input.Email, Password: hashedPassword}
    if err := database.DB.Create(&user).Error; err != nil {
        return c.Status(fiber.StatusBadRequest).JSON(fiber.Map{"error": err.Error()})
    }
    token, err := utils.GenerateJWT(user.ID)
    if err != nil {
        return c.Status(fiber.StatusInternalServerError).JSON(fiber.Map{"error": "Failed to generate token"})
    }
    return c.JSON(fiber.Map{"message": "Signup successful", "token": token})
}

func Login(c *fiber.Ctx) error {
    var input LoginInput
    if err := c.BodyParser(&input); err != nil {
        return c.Status(fiber.StatusBadRequest).JSON(fiber.Map{"error": err.Error()})
```

Fig A2.1.3 Backend Code

```go
// Speech (transcription) route
app.Post("/speech", middleware.JWTMiddleware, controllers.Speech)

// Conversation routes (with JWT middleware)
app.Post("/conversations/new", middleware.JWTMiddleware, controllers.NewConversation)
app.Get("/conversations/:user_uuid", middleware.JWTMiddleware, controllers.ListConversations)
app.Get("/conversations/:conversation_id/messages", middleware.JWTMiddleware, controllers.ConversationMessages)

// RAG routes (with JWT middleware)
app.Post("/rag/upload", middleware.JWTMiddleware, controllers.Upload)
app.Post("/rag/query", middleware.JWTMiddleware, controllers.Query)
app.Post("/rag/speech", middleware.JWTMiddleware, controllers.SpeechRag)
app.Post("/rag/add_text", middleware.JWTMiddleware, controllers.AddText)
app.Get("/rag/collections", middleware.JWTMiddleware, controllers.ListCollections)
app.Delete("/rag/collections/:collection_name", middleware.JWTMiddleware, controllers.DeleteCollection)

// New: RAG conversation listing and history
app.Get("/rag/conversations/:user_uuid", middleware.JWTMiddleware, controllers.ListRagConversations)
app.Get("/rag/conversations/:conversation_id/messages", middleware.JWTMiddleware, controllers.RagConversationMessages)

// N8N integrations (with JWT middleware)
grp := app.Group("/integrations", middleware.JWTMiddleware)
grp.Post("/deep-research", controllers.DeepResearch)
grp.Post("/sdxl", controllers.SDXL)
grp.Post("/flux", controllers.Flux)
grp.Post("/music-gen", controllers.MusicGen)
grp.Post("/fluxkontext-1", controllers.FluxContextOne)
grp.Post("/fluxkontext-2", controllers.FluxContextTwo)
grp.Post("/video-gen", controllers.VideoGen)
grp.Post("/voice-gen", controllers.VoiceGen)

// Agent Chat endpoints (with JWT middleware)
agent := app.Group("/agent-chat", middleware.JWTMiddleware)
agent.Post("/", controllers.CreateAgentChat)
agent.Get("/:id/outputs", controllers.ListAgentChatOutputs)

// AgentConversation endpoints
agentConv := app.Group("/agent-conversations", middleware.JWTMiddleware)
agentConv.Post("/new", controllers.NewAgentConversation)
agentConv.Get("/:user_uuid", controllers.ListAgentConversations)
agentConv.Get("/:agent_conversation_id/messages", controllers.AgentConversationMessages)

// Library routes
app.Get("/library/content", middleware.JWTMiddleware, controllers.LibraryContent)
```

Fig A2.1.4 Backend Code

APPENDIX 3

A3.1 SAMPLE RAG CODE

```
from fastapi import FastAPI, UploadFile, File, Form, Body, Query
from pydantic import BaseModel
from typing import Optional
import os
import tempfile
from dotenv import load_dotenv
import hashlib

from retrieve import Retriever
from generator_local import LocalGenerator
from generator_api import APIGenerator
from risk_detector import RiskDetector
from ingest import Ingestor


load_dotenv()

app = FastAPI()

EMBED_MODEL = os.getenv("NMINDS_EMBED_MODEL", "all-MiniLM-L6-v2")
USE_LOCAL = os.getenv("NMINDS_USE_LOCAL_LLM", "true").lower() == "true"
LOCAL_LLM_PATH = os.getenv("NMINDS_LLM_LOCAL_PATH", "./models/mistral-7b-gguf/")

CHROMA_DIR = os.getenv('NMINDS_CHROMA_DIR', './chroma_db')
retriever = Retriever(embedding_model_name=EMBED_MODEL, chroma_dir=CHROMA_DIR)
risk_detector = RiskDetector()
local_generator = LocalGenerator(model_path=LOCAL_LLM_PATH) if USE_LOCAL else None
api_generator = APIGenerator()  # fallback
ingestor = Ingestor(embedding_model_name=EMBED_MODEL, chroma_dir=CHROMA_DIR)

# Log status
try:
    lg_backend = local_generator.backend if local_generator else 'none'
    lg_avail = bool(local_generator and local_generator.available)
except Exception:
    lg_backend = 'unknown'
    lg_avail = False
print(f'NMINDS STARTUP: embed={EMBED_MODEL}, use_local={USE_LOCAL}, local_avail={lg_avail},
local_backend={lg_backend}')
print(f'NMINDS STARTUP: api_provider={os.getenv("NMINDS_API_PROVIDER", "gemini")},
```

```
gemini_key_present={bool(os.getenv("GEMINI_API_KEY"))}')

class AskRequest(BaseModel):
    user_id: str
    query: str
    persist: bool = True
    summarize_after: int = 5

@app.post('/ingest')
async def ingest(file: UploadFile = File(None), text: Optional[str] = Form(None), collection: str = Form('main_db')):
    if file:
        contents = await file.read()
        # write temp file in a cross-platform safe location
        tmp = tempfile.NamedTemporaryFile(delete=False)
        try:
            tmp.write(contents)
            tmp.flush()
            tmp_path = tmp.name
        finally:
            tmp.close()
        result = ingestor.ingest_file(tmp_path, collection_name=collection)
        try:
            os.remove(tmp_path)
        except Exception:
            pass
        return {"status": "ok", "source": file.filename, "collection": collection, "result": result}
    elif text:
        result = ingestor.ingest_text(text, collection_name=collection)
        return {"status": "ok", "source": "text", "collection": collection, "result": result}
    else:
        return {"status": "error", "detail": "No input provided"}

@app.post('/ingest/{user_id}')
async def ingest_user(user_id: str, file: UploadFile = File(None), text: Optional[str] = Form(None)):
    """Ingest into a per-user collection named `user_{user_id}`. Accepts file upload or raw text."""
    collection = f'user_{user_id}'
    if file:
        contents = await file.read()
        # write temp file in a cross-platform safe location
        tmp = tempfile.NamedTemporaryFile(delete=False)
        try:
            tmp.write(contents)
            tmp.flush()
            tmp_path = tmp.name
        finally:
            tmp.close()
```

```
        result = ingestor.ingest_file(tmp_path, collection_name=collection)
        try:
            os.remove(tmp_path)
        except Exception:
            pass
        return {"status": "ok", "source": file.filename, "collection": collection, "result": result}
    elif text:
        result = ingestor.ingest_text(text, collection_name=collection)
        return {"status": "ok", "source": "text", "collection": collection, "result": result}
    else:
        return {"status": "error", "detail": "No input provided"}

@app.post('/ask')
async def ask(req: AskRequest):
    # retrieve top 4 documents
    docs = retriever.retrieve(query=req.query, user_id=req.user_id, k=4)
    # risk detection
    risk = risk_detector.evaluate(req.query, docs)
    # select generator
    generator = local_generator if (local_generator and local_generator.available) else api_generator
    prompt = """
You are a compassionate mental health support assistant. Use retrieved context and respond empathetically.

Context:
"""
    combined_context = "\n\n".join([d['text'] for d in docs])
    final_prompt = f"{prompt}\n{combined_context}\nUser: {req.query}\nAssistant:"
    response = generator.generate(final_prompt)

    user_collection = f"user_{req.user_id}"
    if req.persist:
        try:
            # store with role metadata by using ingest_text wrapper which now supports metadata
            ingestor.ingest_text(req.query, collection_name=user_collection)
            ingestor.ingest_text(response, collection_name=user_collection)
        except Exception:
            pass

    # Per configuration: do NOT store chat messages into main_db here.
    # Chats (user messages and assistant replies) are stored only in the user's
    # collection `user_<id>` when `persist` is True. Main DB is reserved for
    # canonical documents and ingest operations via /ingest.

    # Summarization is performed only on explicit request via GET /retrieve/{user_id}.

    return {"answer": response, "risk": risk}
```

```
@app.post('/retrieve')
async def retrieve_endpoint(req: AskRequest):
    """Return the RAG retrieval results (documents + metadata) for a query and user.

    Body: {"user_id": "...", "query": "..."}
    """
    # Return documents only from main_db (no user data)
    q_emb = retriever.embedder.encode(req.query).tolist()
    try:
        main_col = retriever.client.get_collection('main_db')
    except Exception:
        try:
            main_col = retriever.client.create_collection('main_db')
        except Exception:
            return {"documents": []}

    try:
        # request top 4 documents only
        res = main_col.query(query_embeddings=[q_emb], n_results=4)
        docs = []
        # handle responses shaped as dicts
        if isinstance(res, dict):
            docs_list = res.get('documents')
            dists = res.get('distances') or res.get('scores')
            if docs_list and isinstance(docs_list, list):
                docs_rows = docs_list[0] if isinstance(docs_list[0], list) else docs_list
                for idx, d in enumerate(docs_rows):
                    score = None
                    try:
                        if dists:
                            dd = dists[0] if isinstance(dists[0], list) else dists
                            score = dd[idx]
                    except Exception:
                        score = None
                    docs.append({"text": d, "score": score, "source": "main_db"})
        return {"documents": docs}
    except Exception:
        return {"documents": []}

@app.post('/detector')
async def detector_endpoint(req: AskRequest):
    """Run the risk detector on the query combined with retrieved context and
    return only the risk analysis JSON.
    """
    docs = retriever.retrieve(query=req.query, user_id=req.user_id, k=4)
```

```
    risk = risk_detector.evaluate(req.query, docs)
    return {"risk": risk}

@app.get('/list_users')
async def list_users():
    """Return a list of user_ids (collections named user_<id>)"""
    try:
        cols = retriever.client.list_collections()
    except Exception:
        # fallback: try ingest's client
        try:
            cols = ingestor.client.list_collections()
        except Exception:
            return {"users": []}

    user_ids = []
    for c in cols:
        name = None
        try:
            name = c.name
        except Exception:
            try:
                name = c.get('name')
            except Exception:
                name = str(c)
        if isinstance(name, str) and name.startswith('user_'):
            user_ids.append(name[len('user_'):])
    return {"users": user_ids}

@app.delete('/clear_user')
async def clear_user(user_id: str = Form(None), user_id_q: str = Query(None), body: dict = Body(None)):
    """Delete a user's collection and any summary collection/backups.

    Accepts `user_id` via form-data, query param, or JSON body: {"user_id": "..."}.
    """
    # resolve user_id from form -> query -> json body
    if user_id:
        uid = user_id
    elif user_id_q:
        uid = user_id_q
    elif body and isinstance(body, dict):
        uid = body.get('user_id')
    else:
        return {"status": "error", "detail": "user_id required"}

    user_collection = f"user_{uid}"
```

```
    summary_collection = f"user_summary_{uid}"
    try:
        try:
            retriever.client.delete_collection(user_collection)
        except Exception:
            try:
                ingestor.client.delete_collection(user_collection)
            except Exception:
                pass
        try:
            retriever.client.delete_collection(summary_collection)
        except Exception:
            try:
                ingestor.client.delete_collection(summary_collection)
            except Exception:
                pass
    except Exception:
        return {"status": "error"}
    return {"status": "ok"}

@app.delete('/clear_user/{user_id}')
async def clear_user_path(user_id: str):
    """Path-based clear user endpoint: DELETE /clear_user/{user_id}"""
    user_collection = f"user_{user_id}"
    summary_collection = f"user_summary_{user_id}"
    try:
        try:
            retriever.client.delete_collection(user_collection)
        except Exception:
            try:
                ingestor.client.delete_collection(user_collection)
            except Exception:
                pass
        try:
            retriever.client.delete_collection(summary_collection)
        except Exception:
            try:
                ingestor.client.delete_collection(summary_collection)
            except Exception:
                pass
    except Exception:
        return {"status": "error"}
    return {"status": "ok"}

@app.delete('/clear_main')
async def clear_main():
```

```
    """Delete the `main_db` collection and all its data. Use with caution."""
    try:
        try:
            retriever.client.delete_collection('main_db')
        except Exception:
            try:
                ingestor.client.delete_collection('main_db')
            except Exception:
                # If delete_collection not available, try to recreate empty collection
                try:
                    # attempt to drop by creating a new empty collection (best-effort)
                    ingestor.client.create_collection('main_db')
                except Exception:
                    pass
    except Exception:
        return {"status": "error"}
    return {"status": "ok"}

@app.get('/retrieve/{user_id}')
async def retrieve_user_summary(user_id: str):
    """Return the user's summary (user_summary_{id}) if exists; otherwise produce a summary on-the-fly."""
    summary_col = f"user_summary_{user_id}"
    try:
        sc = ingestor.client.get_collection(summary_col)
    except Exception:
        try:
            sc = ingestor.client.create_collection(summary_col)
        except Exception:
            sc = None

    # If summary collection exists and has content, return the latest summary
    if sc is not None:
        try:
            data = sc.get()
            docs = data.get('documents') if isinstance(data, dict) else None
            if docs:
                docs_list = docs[0] if isinstance(docs[0], list) else docs
                if len(docs_list) > 0:
                    return {"summary": docs_list[-1]}
        except Exception:
            pass

    # No precomputed summary: generate on-the-fly from last N user messages
    user_col_name = f"user_{user_id}"
    try:
        uc = ingestor.client.get_collection(user_col_name)
```

```
    except Exception:
        try:
            uc = ingestor.client.create_collection(user_col_name)
        except Exception:
            uc = None

    if uc is None:
        return {"summary": None}

    try:
        data = uc.get()
        docs = data.get('documents') if isinstance(data, dict) else None
        if docs:
            docs_list = docs[0] if isinstance(docs[0], list) else docs
            last_texts = '\n'.join(docs_list[-5:])
            summary_prompt = f"Summarize the user's recent messages and extract user characteristics as
JSON.\n\n{last_texts}\n\nReturn: summary text and JSON with keys: age, tone, preferences, concerns."
            generator = local_generator if (local_generator and local_generator.available) else api_generator
            summ = generator.generate(summary_prompt, max_tokens=300)
            # Do NOT store the generated summary automatically; return it directly.
            return {"summary": summ}
    except Exception:
        return {"summary": None}

if __name__ == '__main__':
    import uvicorn
    uvicorn.run(app, host='0.0.0.0', port=8000)
```

CHAPTER 7

REFERENCES

[1] Brown, T. et al. (2020). Language Models are Few-Shot Learners. Advances in Neural Information Processing Systems (NeurIPS), 33, 1877–1901.

[2] Lewis, P., Perez, E., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS). https://doi.org/10.48550/arXiv.2005.11401

[3] Vaswani, A. et al. (2017). Attention is All You Need. Advances in Neural Information Processing Systems, 30. https://doi.org/10.48550/arXiv.1706.03762

[4] Zhou, Z., et al. (2021). Conversational AI in Mental Health: Current Advances and Challenges. Frontiers in Digital Health, 3:657915. https://doi.org/10.3389/fdgth.2021.657915

[5] Kumar, A., et al. (2022). AI-Based Mental Health Chatbots: A Review of Applications and Ethical Concerns. Journal of Medical Systems, 46(10):78. https://doi.org/10.1007/s10916-022-01832-5

[6] Schulman, J., et al. (2022). The Emergence of Conversational Agents for Therapy and Support. Nature Digital Medicine, 5(1), 42–59.

[7] Gao, J., et al. (2023). Hybrid Memory Architectures for Context-Aware Conversational AI. IEEE Transactions on Neural Networks and Learning Systems, 34(8), 10328–10341.

[8] Google DeepMind. (2024). Gemini 1.5 Technical Report. DeepMind Research. https://deepmind.google/gemini

[9] Kumar, P., & Singh, R. (2022). Retrieval-Augmented Conversational Agents using ChromaDB and Vector Indexing. International Journal of AI Research, 9(4), 214–228.

[10] Koehler, S., et al. (2023). Ethical Design of AI-Powered Therapeutic Chatbots: Safety, Transparency, and Human Oversight. Journal of Ethics in AI, 2(1), 55–74.